INSTITUTO TECNOLÓGICO DE CULIACÁN

Ingeniería en Sistemas Computacionales

Inteligencia Artificial ***18:00 – 19:00***

Docente: José Mario Ríos Félix

Alumna: 23170354 – Valenzuela Duarte Hannia Lucero

TAREA – PUZZLE 5X5

Culiacán, Sinaloa a 02 de Marzo de 2026

Código

```java
package ia;

import java.util.*;

public class IDAStarSolver {
    private long nodosExpandidos;
    private boolean usarConflictoLineal;
    private final int[] dr = {-1, 1, 0, 0};
    private final int[] dc = {0, 0, -1, 1};
    private final String[] nombres = {"Arriba", "Abajo", "Izquierda",
"Derecha"};

    public void resolver(int[] inicial, boolean usarConflicto) {
        this.usarConflictoLineal = usarConflicto;
        this.nodosExpandidos = 0;
        int hInicial = NodoUtil.calcularHeuristica(inicial,
usarConflictoLineal);
        int vacio = -1;
        for(int i = 0; i < inicial.length; i++) if(inicial[i] == 0) vacio = i;

        Nodo raiz = new Nodo(inicial, vacio, 0, hInicial, null, "Inicio");
        int umbral = hInicial;
        long inicioTime = System.currentTimeMillis();

        System.out.println("Buscando solución...");

        while (true) {
            Object resultado = buscar(raiz, umbral);
            if (resultado instanceof Nodo) {
                imprimirSolucion((Nodo) resultado, inicioTime);
                return;
            }
            if ((int)resultado == Integer.MAX_VALUE) return;
            umbral = (int) resultado;
        }
    }

    private Object buscar(Nodo n, int umbral) {
        nodosExpandidos++;
        if (n.getF() > umbral) return n.getF();
        if (n.h == 0) return n;

        int min = Integer.MAX_VALUE;
        int r = n.vacioPos / 5;
        int c = n.vacioPos % 5;

        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nr < 5 && nc >= 0 && nc < 5) {
                int nPos = nr * 5 + nc;
                int[] nuevoTablero = n.tablero.clone();
                nuevoTablero[n.vacioPos] = nuevoTablero[nPos];
                nuevoTablero[nPos] = 0;
```

```java
                if (n.padre != null && Arrays.equals(nuevoTablero,
n.padre.tablero)) continue;

                Nodo hijo = new Nodo(nuevoTablero, nPos, n.g + 1,
                                     NodoUtil.calcularHeuristica(nuevoTablero,
usarConflictoLineal), n, nombres[i]);

                Object res = buscar(hijo, umbral);
                if (res instanceof Nodo) return res;
                if ((int)res < min) min = (int)res;
            }
        }
        return min;
    }

    private void imprimirSolucion(Nodo n, long t) {
        List<Nodo> camino = new ArrayList<>();
        Nodo temp = n;
        while (temp != null) {
            camino.add(temp);
            temp = temp.padre;
        }
        Collections.reverse(camino);

        System.out.println("\n>>> EJECUTANDO SOLUCIÓN ÓPTIMA <<<");
        for (int i = 0; i < camino.size(); i++) {
            Nodo paso = camino.get(i);
            System.out.println("\nPASO #" + i + " | Acción: " + paso.movimiento
+ " | Costo acumulado (g): " + paso.g);
            imprimirTableroFormateado(paso.tablero);
            try { Thread.sleep(400); } catch (Exception e) {}
        }

        System.out.println("\n--- RESUMEN DE RENDIMIENTO ---");
        System.out.println("Costo Total del Camino (g): " + n.g);
        System.out.println("Nodos Expandidos: " + nodosExpandidos);
        System.out.println("Tiempo: " + (System.currentTimeMillis() - t) + "
ms");
    }

    private void imprimirTableroFormateado(int[] t) {
        for (int i = 0; i < 25; i++) {
            String pieza = (t[i] == 0) ? "[  ]" : String.format("%3d", t[i]);
            System.out.print(pieza + (i % 5 == 4 ? "\n" : " "));
        }
    }
}
```

```java
package ia;

public class Nodo {
    int[] tablero;
    int vacioPos;
    int g; // Costo acumulado
    int h; // Valor heurístico
    Nodo padre;
    String movimiento;

    public Nodo(int[] tablero, int vacioPos, int g, int h, Nodo padre, String
movimiento) {
        this.tablero = tablero.clone();
        this.vacioPos = vacioPos;
        this.g = g;
        this.h = h;
        this.padre = padre;
        this.movimiento = movimiento;
    }

    public int getF() {
        return g + h;
    }
}
```

```java
package ia;

public class NodoUtil {
          public static final int SIZE = 5;

          public static int calcularHeuristica(int[] tiles, boolean
usarConflictoLineal) {
              int m = 0;
              // 1. Distancia de Manhattan
              for (int i = 0; i < tiles.length; i++) {
                  if (tiles[i] != 0) {
                      int target = tiles[i] - 1;
                      m += Math.abs(i / SIZE - target / SIZE) + Math.abs(i %
SIZE - target % SIZE);
                  }
              }

              // 2. Conflicto Lineal (Penalización por piezas en orden inverso)
              int lc = 0;
              for (int r = 0; r < SIZE; r++) {
                  for (int i = 0; i < SIZE; i++) {
                      for (int j = i + 1; j < SIZE; j++) {
                          int vI = tiles[r * SIZE + i];
                          int vJ = tiles[r * SIZE + j];
```

```
                        if (vI != 0 && vJ != 0 && (vI-1)/SIZE == r && (vJ-
1)/SIZE == r && vI > vJ)
                            lc += 2;
                    }
                }
            }
            return m + lc;
        }

        public static boolean esSoluble(int[] tiles) {
            int inv = 0;
            for (int i = 0; i < tiles.length; i++) {
                for (int j = i + 1; j < tiles.length; j++) {
                    if (tiles[i] != 0 && tiles[j] != 0 && tiles[i] >
tiles[j]) inv++;
                }
            }
            return inv % 2 == 0;
        }
    }
```

```
package ia;

import java.util.Scanner;

import java.util.Random;


public class Principal {

   private static int[] tableroActual = null;


   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      IDAStarSolver solver = new IDAStarSolver();

      int opcion;


      do {

         System.out.println("\n===== 24 PUZZLE - CONFIGURACIÓN =====");

         System.out.println("1. Generar tablero FÁCIL (20 movimientos)");

         System.out.println("2. Generar tablero DIFÍCIL (100 movimientos)");
```

```
        System.out.println("3. Resolver con Manhattan");
        System.out.println("4. Resolver con Manhattan + Conflicto Lineal");
        System.out.println("5. Salir");
        System.out.print("Seleccione una opción: ");
        opcion = sc.nextInt();

        switch (opcion) {
            case 1:
                tableroActual = generarTablero(20);
                System.out.println("\nTablero FÁCIL generado:");
                imprimir(tableroActual);
                break;
            case 2:
                tableroActual = generarTablero(100);
                System.out.println("\nTablero DIFÍCIL generado:");
                imprimir(tableroActual);
                break;
            case 3:
                if (tableroActual != null) solver.resolver(tableroActual, false);
                else System.out.println("Genera un tablero primero.");
                break;
            case 4:
                if (tableroActual != null) solver.resolver(tableroActual, true);
                else System.out.println("Genera un tablero primero.");
                break;
        }
    } while (opcion != 5);
    sc.close();
}
```

```
    private static int[] generarTablero(int pasos) {
        int[] t = new int[25];
        for(int i=0; i<24; i++) t[i] = i+1;
        t[24] = 0;
        int vacio = 24;
        Random r = new Random();
        for(int i=0; i<pasos; i++) {
            int dir = r.nextInt(4);
            int nr = (vacio/5) + (dir==0?-1:dir==1?1:0);
            int nc = (vacio%5) + (dir==2?-1:dir==3?1:0);
            if(nr>=0 && nr<5 && nc>=0 && nc<5) {
                int next = nr*5 + nc;
                t[vacio] = t[next]; t[next] = 0; vacio = next;
            }
        }
        return t;
    }

    private static void imprimir(int[] t) {
        for(int i=0; i<25; i++) System.out.print((t[i]<10?" ":"") + t[i] + (i%5==4?"\n":" "));
    }
}
```

Tabla Comparativa

Métrica de Rendimiento	Configuración FÁCIL (Caso Real)	Configuración DIFÍCIL (Proyección)
Longitud de la solución (g)	**16 - 20 movimientos**	**100 movimientos**
Nodos Expandidos	**56 nodos**	**> 15,000 nodos** (estimado)
Tiempo de ejecución	**6,987 ms**	**> 120,000 ms** (2+ minutos)
Costo Computacional	**Bajo:** La heurística guía al algoritmo casi directamente a la meta.	**Muy Alto:** El algoritmo debe explorar múltiples ramas antes de confirmar la ruta óptima.
Uso de Memoria (RAM)	**Mínimo:** Pocos estados almacenados en la lista abierta.	**Considerable:** Miles de estados guardados para evitar ciclos y re-evaluar rutas.

Como conclusión de este ejercicio, me di cuenta de que usar IDA* fue la mejor decisión para un tablero de 5x5, porque al ser un espacio de estados gigante, este algoritmo evita que la computadora se trabe por falta de memoria. El hecho de que se haya resuelto en 16 pasos expandiendo solo 56 nodos me confirma que la lógica para guiar la búsqueda está súper bien ajustada, ya que casi no dio vueltas innecesarias. Aunque los 6,987 ms parezcan algo de tiempo, la verdad es que el programa se tarda más imprimiendo cada tablita en la consola de Eclipse que haciendo los cálculos. Para el nivel difícil de 100 movimientos, este método es clave porque permite seguir buscando la ruta óptima paso a paso sin que el sistema colapse por guardar tanta información.